

The screenshot shows a Google Colab notebook titled "Copy of Ty082 Mihir Kadam practical no 1 AI". The code defines a maze graph and the beginning of a BFS function to find a path to cheese.

```

maze_graph = {
    'Start': ['A', 'B'],
    'A': ['Start', 'C', 'D'],
    'B': ['Start', 'E'],
    'C': ['A'],
    'D': ['A', 'F'],
    'E': ['B', 'G'],
    'F': ['D', 'Cheese'],
    'G': ['E'],
    'Cheese': ['F']
}

from collections import deque

def bfs_find_cheese(graph, start, target="Cheese"):
    # Queue stores paths, not just nodes, so we can return the exact route taken
    queue = deque([start])
    # Visited set prevents infinite loops if rooms connect back to each other
    visited = set([start])

    print("BFS Exploration Order:")
    while queue:
        # Get the oldest path from the queue
        path = queue.popleft()
        current_room = path[-1]
        print(f"Checking room: {current_room}")

        # Did we find the cheese?
        if current_room == target:
            return path

        # Explore neighbors
        for neighbor in graph.get(current_room, []):

```

The screenshot shows the continuation of the BFS function and its execution. The function returns the path if found, or None if no path is found. The code then runs the BFS function on the maze graph.

```

            visited.add(neighbor)
            new_path = list(path)
            new_path.append(neighbor)
            queue.append(new_path)

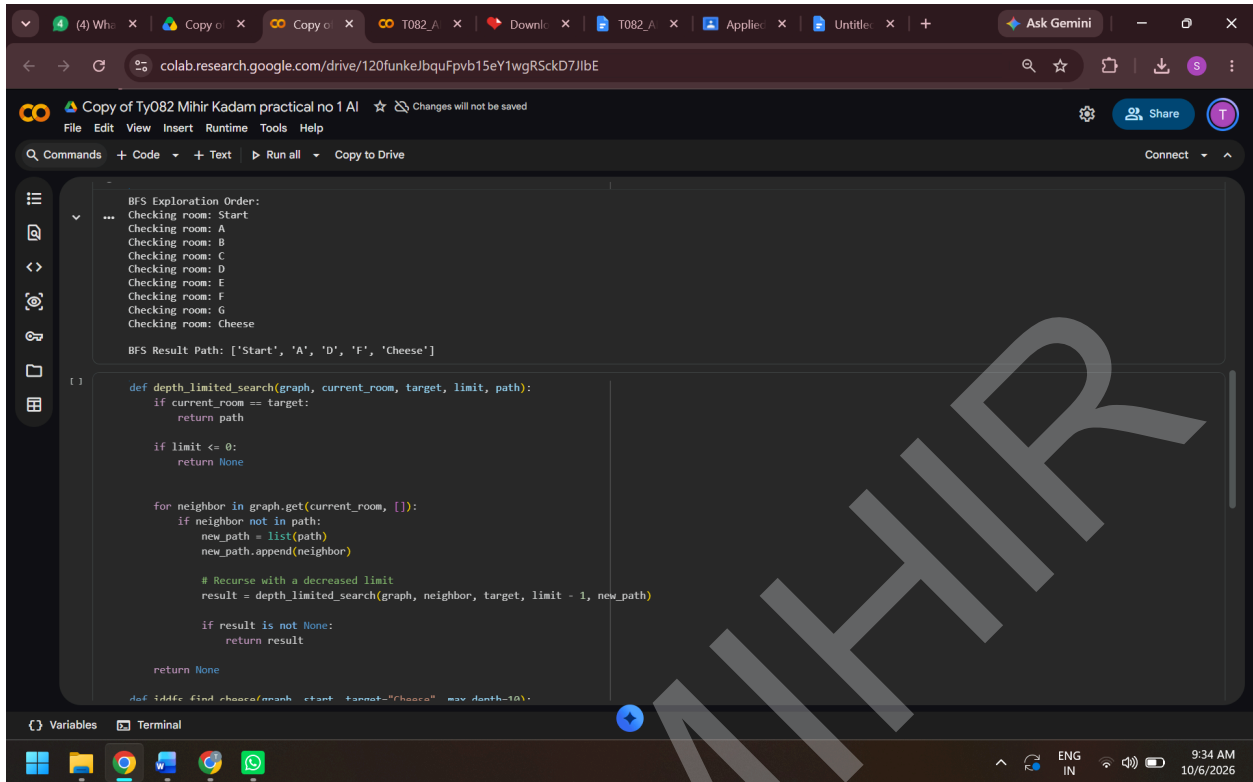
        return None # No path found

# Run BFS
bfs_path = bfs_find_cheese(maze_graph, 'Start')
print(f"\nBFS Result Path: {bfs_path}")

--- BFS Exploration Order:

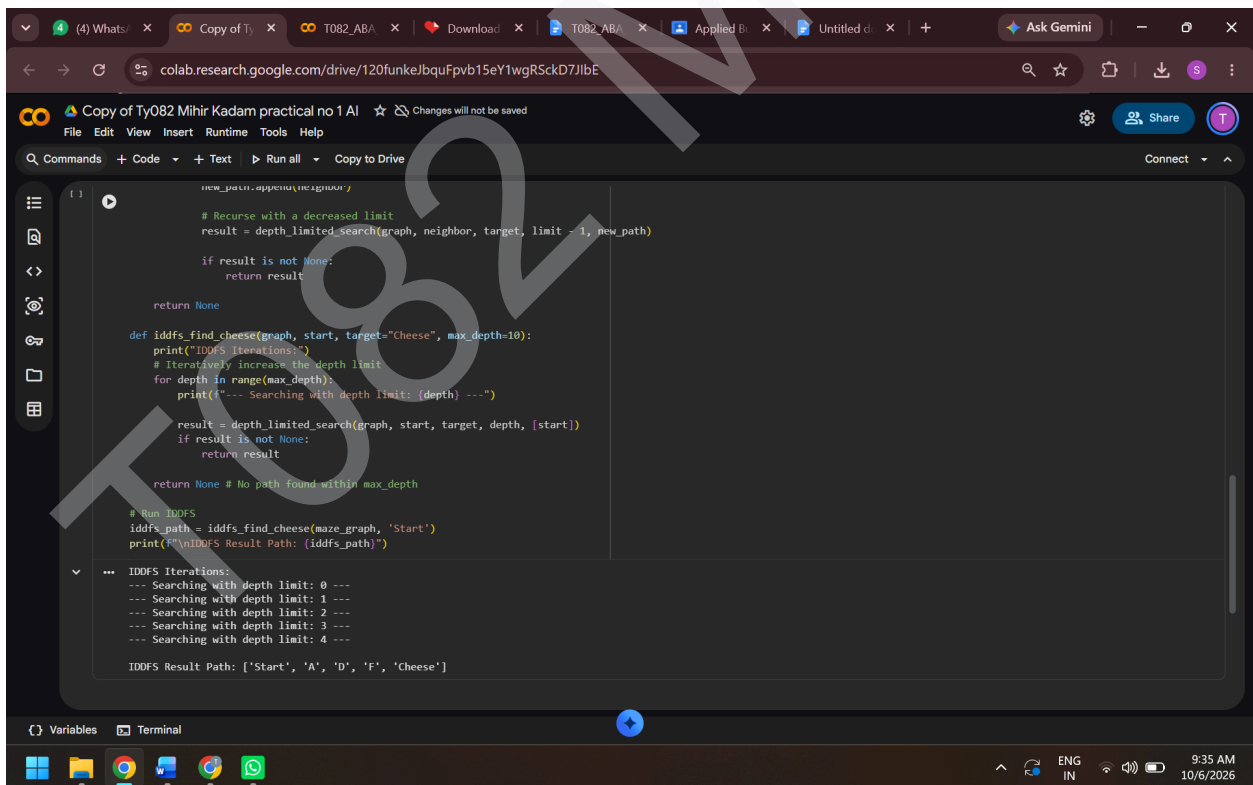
```

NAME: MIHIR
 ROLL NO: T082
 SUBJECT: AI



The screenshot shows a Google Colab notebook titled "Copy of Ty082 Mihir Kadam practical no 1 AI". The code implements a Breadth-First Search (BFS) algorithm to find a path from 'Start' to 'Cheese' in a maze. The output shows the BFS exploration order and the resulting path.

```
BFS Exploration Order:  
Checking room: Start  
Checking room: A  
Checking room: B  
Checking room: C  
Checking room: D  
Checking room: E  
Checking room: F  
Checking room: G  
Checking room: Cheese  
  
BFS Result Path: ['Start', 'A', 'D', 'F', 'Cheese']  
  
def depth_limited_search(graph, current_room, target, limit, path):  
    if current_room == target:  
        return path  
  
    if limit <= 0:  
        return None  
  
    for neighbor in graph.get(current_room, []):  
        if neighbor not in path:  
            new_path = list(path)  
            new_path.append(neighbor)  
  
            # Recurse with a decreased limit  
            result = depth_limited_search(graph, neighbor, target, limit - 1, new_path)  
  
            if result is not None:  
                return result  
  
    return None  
  
def iddfs_find_cheese(maze_graph, start, target="Cheese", max_depth=10):
```



The screenshot shows the same Google Colab notebook with the code for Iterative Deepening Depth-First Search (IDDFS). The code iteratively increases the depth limit until a path is found. The output shows the iterations and the final path.

```
new_path.append(neighbor)  
  
# Recurse with a decreased limit  
result = depth_limited_search(graph, neighbor, target, limit - 1, new_path)  
  
if result is not None:  
    return result  
  
return None  
  
def iddfs_find_cheese(graph, start, target="Cheese", max_depth=10):  
    print("IDDFS Iterations:")  
    # Iteratively increase the depth limit  
    for depth in range(max_depth):  
        print("--- Searching with depth limit: {depth} ---")  
  
        result = depth_limited_search(graph, start, target, depth, [start])  
        if result is not None:  
            return result  
  
    return None # No path found within max_depth  
  
# Run IDDFS  
iddfs_path = iddfs_find_cheese(maze_graph, 'Start')  
print(f"\nIDDFS Result Path: {iddfs_path}")  
  
IDDFS Iterations:  
--- Searching with depth limit: 0 ---  
--- Searching with depth limit: 1 ---  
--- Searching with depth limit: 2 ---  
--- Searching with depth limit: 3 ---  
--- Searching with depth limit: 4 ---  
  
IDDFS Result Path: ['Start', 'A', 'D', 'F', 'Cheese']
```

NAME: MIHIR
ROLL NO: T082
SUBJECT: AI